RISC-V 模拟器运行用户程序的几种方法

Mingzhu Yan

2026-01-28

Outline

1. 运行用户程序 2
2. 系统调用 3
3. 基于 HTIF 协议 4
4. 基于 Semihosting Extension 15
5. 总结 19

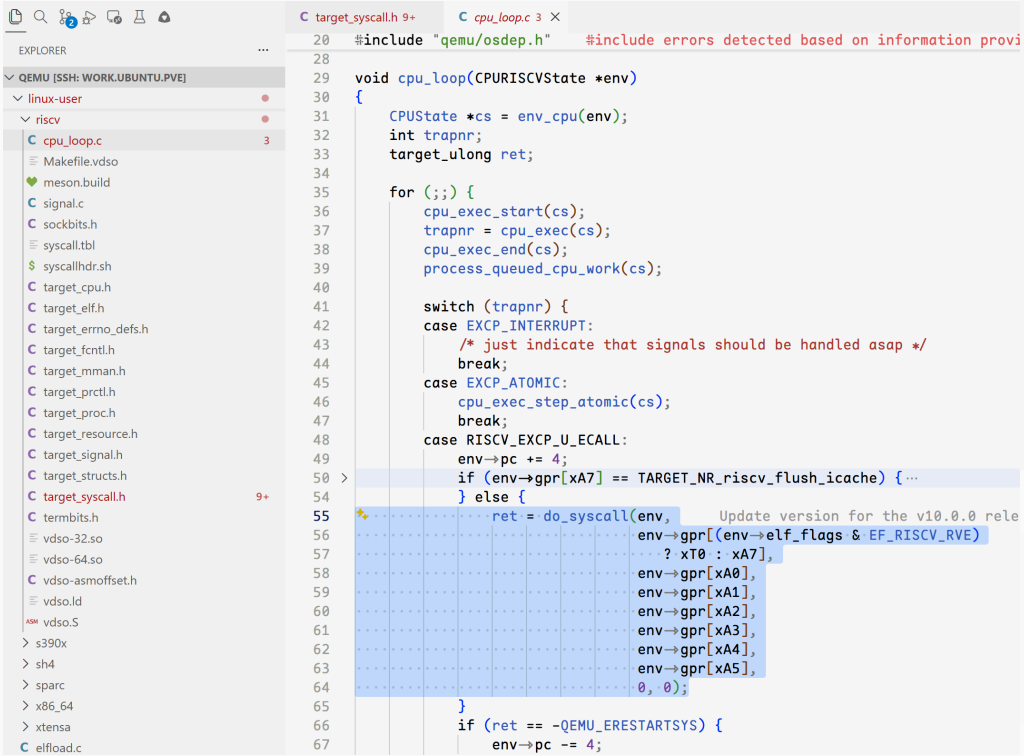
1. 运行用户程序

在 X86 架构的 linux 或者 bsd 上直接运行一个 RISC-V 架构的使用了 printf 的可执行文件

- 系统调用
- HTIF 协议
- Semihosting

2.1 qemu user

qemu-user 可以轻松在 x86-64 系统上运行一个其他架构的用户程序



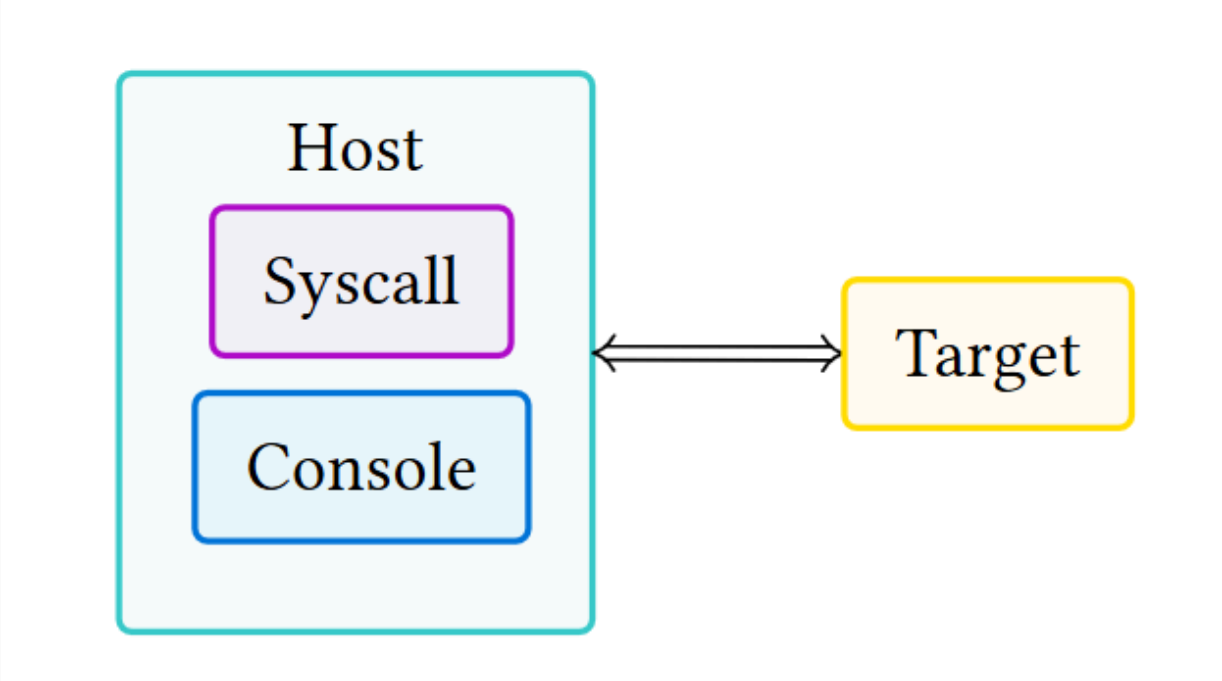

```
case TARGET_NR_write:
    if (arg2 == 0 && arg3 == 0) {
        return get_errno(safe_write(arg1, 0, 0));
    }
    if (!(p = lock_user(VERIFY_READ, arg2, arg3, 1)))
        return -TARGET_EFAULT;
    if (fd_trans_target_to_host_data(arg1)) {
        void *copy = g_malloc(arg3);
        memcpy(copy, p, arg3);
        ret = fd_trans_target_to_host_data(arg1)(copy, arg3);
        if (ret >= 0) {
            ret = get_errno(safe_write(arg1, copy, ret));
        }
        g_free(copy);
    } else {
        ret = get_errno(safe_write(arg1, p, arg3));
    }
    unlock_user(p, arg2, 0);
    return ret;
```

```
case TARGET_FREEBSD_NR_write: /* write(2) */
    ret = do_bsd_write(arg1, arg2, arg3);
    break;
```

3.1 HTIF 协议

https://github.com/Timmmm/htif_spec/releases/download/v0.0.1/htif-0.0.1.pdf

- HTIF consists of two separate 8-byte MMIO devices: tohost and fromhost.



3.2 tohost

Devices

- 0 Syscalls
- 1 Console

Offset (bytes)	Type	Field
0	`u8[6]`	`payload`
6	`u8`	`command`
7	`u8`	`device`

3.2 tohost

Number	Syscall
17	getcwd
25	fcntl
34	mkdirat
35	unlinkat
37	linkat
38	renameat
46	ftruncate
48	faccessat
49	chdir
56	openat
57	close
62	lseek
63	read
64	write
67	pread
68	pwrite
78	readlinkat
79	fstatat
80	fstat
93	exit
291	statx
1039	lstat
2011	getmainvars

Offset	Corresponding Linux ABI Register
0	*syscall number*
1	a0 (arg/return)
2	a1
3	a2
4	a3
5	a4
6	a5
7	-

3.2 tohost

```
function htif_store(Physaddr(paddr), width, data) = {
  let cmd = Mk_htif_cmd(htif_tohost);
  match cmd[device] {
    0x00 => { // syscall-proxy
      if cmd[payload][0] == 0b1 then {
        htif_done = true;
        htif_exit_code = (zero_extend(64, cmd[payload]) >> 1)
      } else {
        dispatch_syscall(cmd[payload]);
      }
    },
    0x01 => { // terminal
      match cmd[cmd] {
        Implement with Codex
        0x00 => /* TODO: terminal input handling */ (),
        0x01 => plat_term_write(cmd[payload][7..0]),
        c    => print("Unknown term cmd: " ^ bits_str(c))
      };
      // reset to ack
      reset_htif()
    },
    _ => print("htif-???? cmd: " ^ bits_str(data))
  }
}
```

3.3 spike 运行用户程序

Spike 也实现了基于 HTIF 的系统调用

```
void syscall_t::handle_syscall(command_t cmd)
{
  if (cmd.payload() & 1) // test pass/fail
  {
    htif->exitcode = cmd.payload();
    if (htif->exit_code())
      std::cerr << "*** FAILED *** (tohost = " << htif->exit_code() << ")" << std::endl;
    return;
  }
  else // proxied system call
    dispatch(cmd.payload());

  cmd.respond(1);
}

reg_t syscall_t::sys_exit(reg_t code, reg_t a1, reg_t a2, reg_t a3, reg_t a4, reg_t a5, reg_t a6
{
  htif->htif_exit(code << 1 | 1);
  return 0;
}
```

```
void handle_trap(trapframe_t* tf)
{
  if ((intptr_t)tf->cause < 0)
    return handle_interrupt(tf);

  typedef void (*trap_handler)(trapframe_t*);

  const static trap_handler trap_handlers[] = {
    [CAUSE_MISALIGNED_FETCH] = handle_misaligned_fetch,
    [CAUSE_FETCH_ACCESS] = handle_instruction_access_fault,
    [CAUSE_LOAD_ACCESS] = handle_load_access_fault,
    [CAUSE_STORE_ACCESS] = handle_store_access_fault,
    [CAUSE_FETCH_PAGE_FAULT] = handle_fault_fetch,
    [CAUSE_ILLEGAL_INSTRUCTION] = handle_illegal_instruction,
    [CAUSE_USER_ECALL] = handle_syscall,      Upgrade to privileged architec
    [CAUSE_BREAKPOINT] = handle_breakpoint,
    [CAUSE_MISALIGNED_LOAD] = handle_misaligned_load,
    [CAUSE_MISALIGNED_STORE] = handle_misaligned_store,
    [CAUSE_LOAD_PAGE_FAULT] = handle_fault_load,
    [CAUSE_STORE_PAGE_FAULT] = handle_fault_store,
    [CAUSE_SOFTWARE_CHECK_FAULT] = handle_software_check_fault,
  };

  kassert(tf->cause < ARRAY_SIZE(trap_handlers) && trap_handlers[tf->cause])

  trap_handler f = (void*)pa2kva(trap_handlers[tf->cause]);

  f(tf);
}
```

一般使用 spike 运行用户程序需要配合 riscv-pk (代理内核)，riscv-pk 实现了基本的 trap_handler，可以将标准的 linux 系统调用转换为 htif 格式的 syscall

spike riscv-pk hello.elf

3.4 pk 到 spike

riscv-pk 会使用 htif 协议实现的 syscall 实现真实的系统调用

```c
ssize_t sys_read(int fd, char* buf, size_t n)
{
  ssize_t r = -EBADF;
  file_t* f = file_get(fd);
  char kbuf[MAX_BUF];

  if (f) {
    for (size_t total = 0; ; ) {
      size_t cur = MIN(n - total, MAX_BUF);
      r = file_read(f, kbuf, cur);
      if (r < 0)
        break;

      memcpy_to_user(buf, kbuf, r);

      total += r;
      buf += r;
      if (r < cur || total == n) {
        r = total;
        break;
      }
    }

    file_decref(f);
  }

  return r;
}
```

```c
ssize_t file_read(file_t* f, void* buf, size_t size)
{
  return frontend_syscall(SYS_read, f->kfd, kva2pa(buf), size, 0, 0, 0, 0);
}
```

3.5 nanoprint

https://github.com/charlesnicholson/nanoprintf

Usage

Add the following code to one of your source files to compile the nanoprintf implementation:

```
// define your nanoprintf configuration macros here (see "Configuration" below)
#define NANOPRINTF_IMPLEMENTATION
#include "path/to/nanoprintf.h"
```

Then, in any file where you want to use nanoprintf, simply include the header and call the npf_ functions:

```
#include "nanoprintf.h"

void print_to_uart(void) {
  npf_pprintf(&my_uart_putc, NULL, "Hello %s%c %d %u %f\n", "worl", 'd', 1, 2, 3.f);
}

void print_to_buf(void *buf, unsigned len) {
  npf_snprintf(buf, len, "Hello %s", "world");
}
```

3.6 sail-model 使用 nanoprint

runtime.c

```c
#include "nanoprintf.h"

extern void htif_putc(int c, void *ctx);

int printf(const char *fmt, ... ) {
  va_list ap;
  va_start(ap, fmt);
  int rc = npf_vpprintf(&htif_putc, NULL, fmt, ap);
  va_end(ap);
  return rc;
}

```

crt0.S

```asm
1:

.global htif_putc
htif_putc:
    la t0, tohost
    sw a0, 0(t0)
    // device=1 (terminal), cmd=1 (output)
    li a0, 0x01010000
    sw a0, 4(t0)
    ret

```

3.7 基于 htif 的缺点

- 系统调用数量众多
- 难以同时兼顾不同的操作系统
- FreeBSD System Calls Table https://alfonsosiciliano.gitlab.io/posts/2021-01-02-freebsd-system-calls-table.html
- Linux kernel syscall tables https://syscalls.mebeim.net/?table=riscv/64/rv64/latest

3.8 FreeBSD System Calls Table

FreeBSD System Calls Table

Posted on January 2, 2021

Important! This post has been rendered obsolete by FreeBSD 15 System Calls Table.

A system call is the “tool” used by a software in the userspace to request a “service” to the kernel, the FreeBSD Operating System provides over 400 system calls. They are defined in syscalls.master, this file is processed by makesyscalls.sh or makesyscalls.lua to create init_sysent.c, syscall.h, and so on.

The system calls list has 576 entries (15: unimplemented, 55: nosys/lkmnosys, 43: compat and 72: old/obsolete) for 13-CURRENT at the time of writing. The entry point of a system call is *sys_{name}*, except *syscall* and *__syscall* described below.

This table is partially auto-generated by scanning *init_sysent.c* and manually filled, indeed its purpose is just a personal study of the FreeBSD kernel.

Number	Name	Short Description	Implementation
0	syscall	Indirect system call	kern/subr_syscall.c
1	exit	Terminate the calling process	kern/kern_exit.c
2	fork	Create a new process	kern/kern_fork.c
3	read	Read input	kern/sys_generic.c
4	write	Write output	kern/sys_generic.c
5	open	Open or create a file for reading, writing or executing	kern/vfs_syscalls.c
6	close	Delete a descriptor	kern/kern_descrip.c
7	wait4	Wait for processes to change status	kern/kern_exit.c
8	[old] creat		
9	link	Make a hard file link	kern/vfs_syscalls.c
10	unlink	Remove directory entry	kern/vfs_syscalls.c
11	[obsolete] execv		
12	chdir	Change current working directory	kern/vfs_syscalls.c

3.9 Linux kernel syscall tables

Linux kernel syscall tables

Architecture: RISC-V 64-bit, RV64 ABI (64-bit) **Kernel version:** latest (v6.17)

Powered by Systrack v0.8 [JSON table] [kernel config] [analysis log] [website source]

						[-] Signature [compact]
Number (a7)		[-] Name	[-] Symbol	[-] Definition location	[-] Kconfig	Arguments (a0, a1, a2, a3, a4, a5)
0	0x0	io_setup	__riscv_sys_io_setup	fs/aio.c:1382	AIO	unsigned nr_events, aio_context_t *ctxp
1	0x1	io_destroy	__riscv_sys_io_destroy	fs/aio.c:1451	AIO	aio_context_t ctx
2	0x2	io_submit	__riscv_sys_io_submit	fs/aio.c:2082	AIO	aio_context_t ctx_id, long nr, struct iocb **iocbpp
3	0x3	io_cancel	__riscv_sys_io_cancel	fs/aio.c:2176	AIO	aio_context_t ctx_id, struct iocb *iocb, struct io_event *result
4	0x4	io_getevents	__riscv_sys_io_getevents	fs/aio.c:2251	AIO	aio_context_t ctx_id, long min_nr, long nr, struct io_event *events, struct __kernel_timespec *timeout
5	0x5	setxattr	__riscv_sys_setxattr	fs/xattr.c:743		const char *pathname, const char *name, const void *value, size_t size, int flags
6	0x6	lsetxattr	__riscv_sys_lsetxattr	fs/xattr.c:750		const char *pathname, const char *name, const void *value, size_t size, int flags
7	0x7	fsetxattr	__riscv_sys_fsetxattr	fs/xattr.c:758		int fd, const char *name, const void *value, size_t size, int flags
8	0x8	getxattr	__riscv_sys_getxattr	fs/xattr.c:888		const char *pathname, const char *name, void *value, size_t size
9	0x9	lgetxattr	__riscv_sys_lgetxattr	fs/xattr.c:894		const char *pathname, const char *name, void *value, size_t size
10	0xa	fgetxattr	__riscv_sys_fgetxattr	fs/xattr.c:901		int fd, const char *name, void *value, size_t size
11	0xb	listxattr	__riscv_sys_listxattr	fs/xattr.c:998		const char *pathname, char *list, size_t size
12	0xc	llistxattr	__riscv_sys_llistxattr	fs/xattr.c:1004		const char *pathname, char *list, size_t size
13	0xd	flistxattr	__riscv_sys_flistxattr	fs/xattr.c:1010		int fd, char *list, size_t size
14	0xe	removexattr	__riscv_sys_removexattr	fs/xattr.c:1097		const char *pathname, const char *name
15	0xf	lremovexattr	__riscv_sys_lremovexattr	fs/xattr.c:1103		const char *pathname, const char *name
16	0x10	fremovexattr	__riscv_sys_fremovexattr	fs/xattr.c:1109		int fd, const char *name
17	0x11	getcwd	__riscv_sys_getcwd	fs/d_path.c:412		char *buf, unsigned long size
19	0x13	eventfd2	__riscv_sys_eventfd2	fs/eventfd.c:423		unsigned int count, int flags
20	0x14	epoll_create1	__riscv_sys_epoll_create1	fs/eventpoll.c:2296	EPOLL	int flags
21	0x15	epoll_ctl	__riscv_sys_epoll_ctl	fs/eventpoll.c:2481	EPOLL	int epfd, int op, int fd, struct epoll_event *event
22	0x16	epoll_pwait	__riscv_sys_epoll_pwait	fs/eventpoll.c:2597	EPOLL	int epfd, struct epoll_event *events, int maxevents, int timeout, const sigset_t *sigmask, size_t sigsetsize
23	0x17	dup	__riscv_sys_dup	fs/file.c:1454		unsigned int fildes
24	0x18	dup3	__riscv_sys_dup3	fs/file.c:1430		unsigned int oldfd, unsigned int newfd, int flags
25	0x19	fcntl	__riscv_sys_fcntl	fs/fcntl.c:576		unsigned int fd, unsigned int cmd, unsigned long arg
26	0x1a	inotify_init1	__riscv_sys_inotify_init1	fs/notify/inotify/inotify_user.c:719	INOTIFY_USER	int flags
27	0x1b	inotify_add_watch	__riscv_sys_inotify_add_watch	fs/notify/inotify/inotify_user.c:729	INOTIFY_USER	int fd, const char *pathname, u32 mask
28	0x1c	inotify_rm_watch	__riscv_sys_inotify_rm_watch	fs/notify/inotify/inotify_user.c:786	INOTIFY_USER	int fd, __s32 wd
29	0x1d	ioctl	__riscv_sys_ioctl	fs/ioctl.c:584		unsigned int fd, unsigned int cmd, unsigned long arg
30	0x1e	ioprio_set	__riscv_sys_ioprio_set	block/ioprio.c:65	BLOCK	int which, int who, int ioprio
31	0x1f	ioprio_get	__riscv_sys_ioprio_get	block/ioprio.c:180	BLOCK	int which, int who
32	0x20	flock	__riscv_sys_flock	fs/locks.c:2135		unsigned int fd, unsigned int cmd
33	0x21	mknodat	__riscv_sys_mknodat	fs/namei.c:4310		int dfd, const char *filename, umode_t mode, unsigned int dev
34	0x22	mkdirat	__riscv_sys_mkdirat	fs/namei.c:4414		int dfd, const char *pathname, umode_t mode
35	0x23	unlinkat	__riscv_sys_unlinkat	fs/namei.c:4684		int dfd, const char *pathname, int flag
36	0x24	symlinkat	__riscv_sys_symlinkat	fs/namei.c:4769		const char *oldname, int newdfd, const char *newname

4. 基于 Semihosting Extension

仅仅支持部分常见的系统调用

- Spec https://github.com/riscv-non-isa/riscv-semihosting
- Gem5 实现 https://github.com/gem5/gem5/blob/stable/src/arch/generic/semihosting.cc
- QEMU 实现 https://github.com/qemu/qemu/blob/master/semihosting/syscalls.c#L153

必须链接到 picolibc https://github.com/picolibc/picolibc

4.1 Semihosting

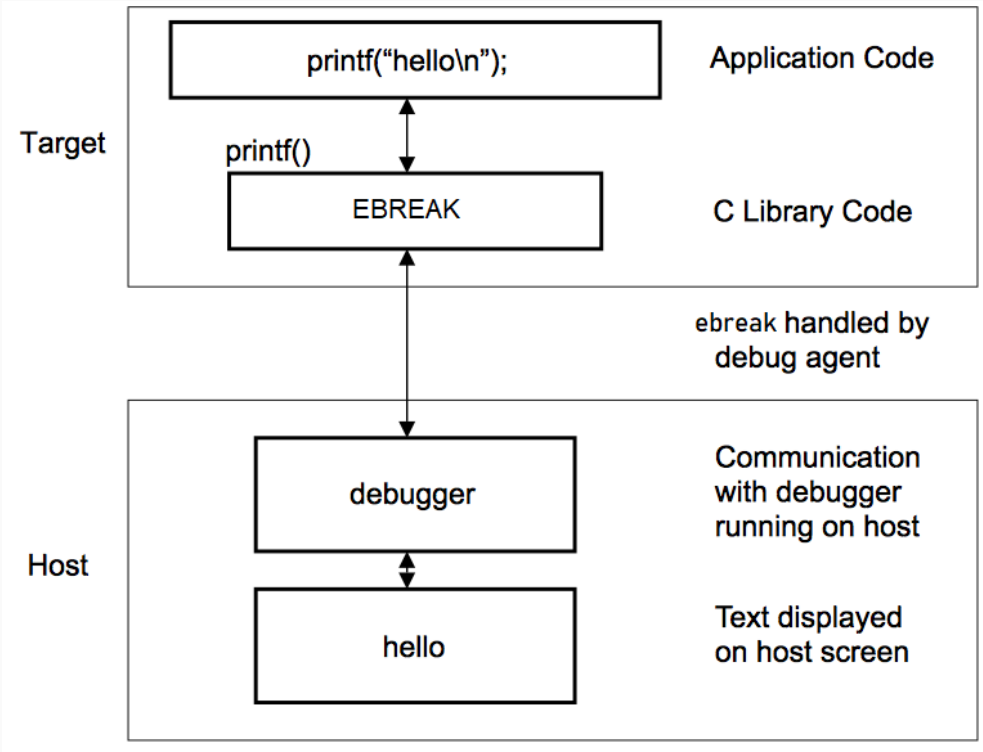
- HTIF 提供了两个 MMIO 设备
- Semihosting 则是特殊的指令序列

```c
uintptr_t
sys_semihost_write(int fd, const void *buf, size_t count)
{
    return sys_semihost3(SYS_WRITE, fd, (sh_param_t)(uintptr_t)buf,
                         count);
}
```

```
slli x0, x0, 0x1f    # 0x01f01013    Entry NOP
ebreak               # 0x00100073    Break to debugger
srai x0, x0, 7       # 0x40705013    Exit NOP
```

```
> powerpc
v riscv
  meson.build
  semihost-riscv.S
> rx
> sh
```

```asm
sys_semihost:
        slli zero, zero, 0x1f
        ebreak
        srai zero, zero, 0x7
        ret
        .option pop
```



6 Semihosting operations

The following semihosting operations are available:

- SYS_CLOCK (0x10)
- SYS_CLOSE (0x02)
- SYS_ELAPSED (0x30)
- SYS_ERRNO (0x13)
- SYS_EXIT (0x18)
- SYS_EXIT_EXTENDED (0x20)
- SYS_FLEN (0x0c)
- SYS_GET_CMDLINE (0x15)
- SYS_HEAPINFO (0x16)
- SYS_ISERROR (0x08)
- SYS_ISTTY (0x09)
- SYS_OPEN (0x01)
- SYS_READ (0x06)
- SYS_READC (0x07)
- SYS_REMOVE (0x0e)
- SYS_RENAME (0x0f)
- SYS_SEEK (0x0a)
- SYS_SYSTEM (0x12)
- SYS_TICKFREQ (0x31)
- SYS_TIME (0x11)
- SYS_TMPNAM (0x0d)
- SYS_WRITE (0x05)
- SYS_WRITEC (0x03)
- SYS_WRITE0 (0x04)

2.2. Semihosting Parameters

The type of semihosting operation and its parameters are specified using general purpose registers. The OPERATION NUMBER is specified in the `a0` register, and the PARAMETER is specified in the `a1` register, whereas the RETURN VALUE is available in the `a0` register. All registers and data block fields are XLEN wide.

4.2 Semihosting

```
riscv64-unknown-elf-gcc --specs=/path/to/picolibc.specs --oslib=semihost hello.c
```

5. 总结

1. 系统调用翻译
2. HTIF 协议
3. Semihosting

完整的用户空间仿真需要

1. System call translation
2. POSIX signal handling
3. Threading

- https://www.qemu.org/docs/master/user/main.html